CS 8803-O08: Project Phase 3

Santosh Pande
santosh.pande@cc.gatech.edu

Georgia Institute of Technology

Overview

The purpose of Phase 3 is to implement the back end of a compiler along with several compiler optimizations. The back end will convert the IR code generated in Phase 2 into MIPS assembly code that can be run in the SPIM simulator. The primary optimization will involve the implemenation of two different register allocation strategies: Briggs' and naive.

For this phase, your compiler will be provided both a Tiger file and a corresponding IR file as input. Therefore, if you would like to continue using your existing codebase from Phase 2, you can use the Tiger file as input and produce your own IR. Alternately, if you would like to start over with a fresh codebase, you can choose to use the provided IR file as input.

Requirement 1

Accept IR as an input file using the flag `-r` <path/to/file.ir>. In all tests, the compiler will be executed with both of the following command line arguments: `-f file.tiger -r file.ir`. The compiler is free to choose either the Tiger or IR file as the starting point.

1 Register Allocation

Intermediate representations generally contain large numbers of temporary variables to hold the intermediate results of evaluating an expression. The IR also contains instructions for storing variables to memory and loading data from memory into variables. Before generating MIPS instructions, you must allocate the variables (including temporaries) to registers available in MIPS.

The general idea of register allocation is straightforward. At any given point, the processor can hold a finite number of values in its registers, and the values being operated on must be in those registers. If there are not enough registers available in the processor, then we must temporarily store (known as a spill) a variable's value in memory. This will require additional load and store instructions.

There is a large body of knowledge and research surrounding register allocation, and it is a critical phase of compilation. In this project, you will be implementing two separate register allocation algorithms:

- Naïve
- Global Briggs

1.1 Naïve Register Allocation

The naïve register allocation scheme is the simplest and requires no analysis. All local variables are assigned memory locations on the stack. The operands for each instruction are loaded from memory into registers immediately before each instruction is run. The instruction is then executed, and the result is immediately stored back into memory. This scheme is very slow at runtime due to the large number of load and store instructions. However, it is simple and does not require many registers to implement.

Requirement 2

Implement a naïve register allocator. Use this allocator when the `-n` flag is provided.

1.2 Global Briggs' Style Register Allocation

The last exercise in register allocation is implementing a global (whole function) register allocator using the Briggs' optimistic graph coloring algorithm. This requires you to:

1. Perform liveness analysis across the basic blocks in the CFG.
2. Identify the live ranges (webs).
3. Construct the interference graph.
4. Calculate spilling costs for each node.
5. Perform graph coloring using Briggs' optimistic coloring algorithm.
6. Allocate the registers.

This allocation scheme is complex and involved. We recommend that you test each step individually as you go and refer to the video lessons and textbook readings for all the details.

Requirement 3

Implement a global (whole function) Briggs' style graph coloring register allocator. Use this allocator when the `-g` flag is provided.

1.3 Allocation Data Structures

Requirement 4

When the `-c` flag is provided, write the control flow graph to a file in GraphViz DOT format. The output file should have the same name and path as the input file with the extension changed to `.cfg.gv`. The format can be of your own design as long as it is valid Graphviz and illustrates the structure of the CFG.

Requirement 5

When the `-v` flag is provided, write the live variable in and out sets for each program point to a file. The output file should have the same name and path as the input file with the extension changed to `.liveness`.

2 Instruction Selection & Code Generation

Instruction selection is the process of mapping IR instructions to the corresponding architectural dependent assembly instructions. Code generation involves outputting this assembly code in the correct format. For this project, you will be mapping the IR input to MIPS assembly code output suitable to be run in the SPIM simulator.

2.1 MIPS

MIPS is a reduced instruction set computer (RISC) instruction set architecture (ISA). While it is now generally used for education, historically MIPS was used in personal computers, servers, and video game consoles such as the Nintendo 64, Sony PlayStation, PlayStation 2, and the PlayStation Portable.

MIPS has thirty-two general purpose registers each with their own intended use and conventions. We recommend that you take time to familiarize yourself with the MIPS registers, instructions, and examples. You can either use the MIPS function calling convention, which involves placing the first four arguments in registers and passing the remaining arguments on the stack, or implement one of your own design. More information is available in the *CS 8803-O08: MIPS Guide*.

Requirement 6

Perform instruction selection and output valid MIPS assembly to a file suitable to be run in the SPIM interpreter when any one of the register allocation flags `{-n, -g}` is provided. The MIPS output file should have the same name and path as the input file with the extension set according to the register allocator used: `.naive.s`, `.briggs.s`

Info: Tiger global `"static"` variables must be stored in the MIPS data section. Tiger local `"var"` variables must be stored on the stack to support recursion. Function arguments, whether passed in registers or on the stack, should be spilled to stack memory.

2.2 SPIM

SPIM is a MIPS simulator that you will use to run the assembly output by your compiler. SPIM includes a simple 'OS' which provides several useful system calls and handles loading and executing your program. For this project, you will use a patched version of SPIM which collects additional statistics about the running program. The patch file is located in the Vagrant/Docker test environment repository. SPIM is installed automatically with the patch in the test environments. Follow the example below to install SPIM manually:

Command Line

```
$ sudo apt install --assume-yes git make g++ bison flex
$ git clone https://github.gatech.edu/omscs8803-08/spim-keepstats.git
$ cd spim-keepstats
$ patch -p1 < /location/of/spimfixes.patch
$ cd spim
$ make && sudo make install
$ spim -keepstats -f ../helloworld.s
Hello World
Stats -- #instructions:13  #reads:2  #writes:0  #branches:2  #other:9
```

Info: SPIM expects to find a "`main:`" label where your program's statement sequence begins. You must also generate a final instruction in main, "`jr $ra`", to return control to the OS.

3 Reports

3.1 Compiler Optimization Report

A compiler optimization is an algorithm which takes a program and transforms it to produce a semantically equivalent output program that is better in some way. Common optimizations attempt to minimize a program's execution time, memory footprint, storage size, or power consumption. Often, these optimization problems are NP-complete or even undecidable.

The global register allocator you implemented is an example of a compiler optimization. When analyzing the affects of these optimizations, we are interested in two code quality metrics:

- instruction count
- instruction cost

The instruction count is simply the number of MIPS assembly instructions. The instruction cost is a bit more complex. Inside of a processor, certain instructions take longer to process than others. To keep things simple, SPIM tracks only four types of instructions: reads, writes, branches, and other. Because they involve memory accesses, reads and writes are considered the most "expensive". This is followed by the "branch" statements, which can affect local caches, and finally by the baseline "other" instuctions.

The primary goal of the global Brigg's register allocation strategy implemented in this project is to reduce the number of memory accesses in the resulting assembly code compared to the default naïve register allocation strategy.

Requirement 7

Document and analyze the SPIM statistics for each of the benchmark programs using each of the three register allocion methods. To produce statistics, run SPIM with the flag `-keepstats`. Name the report `phase3_analysis.pdf`.

The report should include what results were expected for each benchmark, whether the compiler met expectations, and an explanation if expectations were not met.

3.2 Design Internals Report

Write a concise report detailing the design of your compiler back end. Your report must include, but is not limited to, the following information:

- MIPS Function calling convention implementation
- Design of data structures used to implement each register allocation scheme: naïve, Briggs.
- Class design and general code structure

- Bugs or features that are not operational

Requirement 8

Write design internals report for the compiler back end. Name the report `phase3_design.pdf`.

4 Extra Credit

For extra credit you may implement Superlocal Value Numbering (SVN):

From the CFG, first detect the extended basic block or EBB (see the relevant lesson and book material related to this). Then implement the value numbering algorithm that detects and eliminates the redundancy of expressions within the EBB by starting with the root of the EBB and walking towards the leaves. SVN walks the IR top down in the above order, performs value numbering on IR statements, and simultaneously performs three different types of optimizations:

1. Propagation and folding of constants
2. Removal of lexically identical expressions (commutativity wherever applicable)
3. Removal of value identical expressions that are not lexically identical

Please classify and report the number of each category of optimization performed. You will position this optimization pass before the Global Register Allocation pass. Use SPIM to output dynamic instruction counts and code size with the optimization enabled. Compare them with values seen when the SVN optimization is disabled. Enable this optimization pass using the flag `-x`.

To receive credit, include an example Tiger program that benefits from the optimization and a report detailing the desired information from above. The optimization should only be applied when the flag listed above is provided.

5 Grading

Your project should be submitted to Gradescope for grading. It is important that your project follows the correct build process and naming conventions outlined below.

5.1 Building

The build environment is the same as in Phase 2. In order to facilitate the building and testing of your code, you must include a *Makefile* at the root of your source code directory. Your *Makefile* should compile and build a single output file under the `cs8803_bin` folder. If using Java, the output file should be a JAR file called `tigerc.jar`. If using C++, the output file should be an executable file called `tigerc`.

5.2 Testing

We will test your compiler on multiple test inputs. Some of these test cases will be provided to you before the due date. The test environment is the same as in Phase 2, with the addition of the SPIM simulator.

5.2.1 Gradescope

You can also test your compiler by submitting early to Gradescope. Gradescope will run **make** to generate the executable, and then run a series of automated tests. Note that additional points may be deducted for errors that are not caught by the automated testing. You can submit your compiler to Gradescope as many times as you would like prior to the deadline. It is highly recommended to **submit early and often**.

5.3 Timeline

Start early! Make sure to submit to the auto-grader early and often to check your code. Here is a suggested timeline showing important milestones:

- Week 1-2 : Complete naïve register allocation and code generation.
- Week 3-4 : Complete CFG output, liveness analysis, and Briggs' allocation.

5.4 Deliverables

Your project should be submitted as a .zip file to Gradescope. The .zip file should contain:

- Makefile - Makefile used to build your code
- phase3_analysis.pdf - Register allocator analysis report
- phase3_design.pdf - Design internals report
- src - Directory containing your compiler source code (C++/Java)

5.5 Rubric

Warning: Your submission is expected to compile cleanly from source. If a submission fails to compile, contains pre-built class files or binaries, or artificially bypasses the auto-grader, it can result in a **zero**.

- Instruction selection and code generation **100 points**
  - using naïve register allocation 65 points
  - using global Briggs' register allocation 35 points
- Data structure output **30 points**
  - Control Flow Graph Graphviz output 15 points
  - Liveness Analysis In-Out sets 15 points
- Reports **20 points**
  - Register allocation code analysis 10 points
  - Design internals 10 points
- Extra Credit *10 points*